

PRODUCTION INCIDENT RUNBOOK

Kubernetes + PostgreSQL

*Percona Operator | Node Drains | Pod Rescheduling |
Resizing | Node Replacement*

Document Type	Operational Runbook
Classification	CONFIDENTIAL — Internal & Executive Distribution
Author Role	Lead SME — Kubernetes Operations / PostgreSQL Administration
Clusters Covered	Cluster-A (3 Control Plane / 6 Nodes)
	Cluster-B (5 Control Plane / 10 Nodes)
Database Platform	Percona Operator for PostgreSQL (PerconaPGCluster CR)

TABLE OF CONTENTS

1. Cluster Environment Overview
2. Scenario 1 — Worker Node Kernel Panic & PG Primary Failover
3. Scenario 2 — etcd Disk Pressure Forces CP Node Replacement
4. Scenario 3 — PG Replica OOMKilled, Resource Resize
5. Scenario 4 — Cilium CrashLoop After OS Upgrade
6. Scenario 5 — PgBouncer Scaled to Zero (Cluster-B)
7. Scenario 6 — CSI Volume Plugin Failure
8. Scenario 7 — pgbackupmonitor CronJob Failures (Cluster-A)
9. Scenario 8 — Control Plane Certificate Expiry
10. Scenario 9 — PG Instance 4/5 Not Ready (Cluster-A)
11. Scenario 10 — Rolling K8s Upgrade Across 10 Nodes
12. Scenario 11 — Prometheus OOMKilled
13. Scenario 12 — PVC Full on PG Primary (WAL Accumulation)
14. Scenario 13 — Node NotReady: Kubelet Memory Leak
15. Scenario 14 — PG Operator CrashLoop After CRD Update
16. Scenario 15 — Scheduled Hardware Decommission
17. Scenario 16 — Network Partition Split-Brain
18. Scenario 17 — High Controller-Manager Restarts (Cluster-B)
19. Scenario 18 — PgBouncer Connection Saturation
20. Scenario 19 — Cordon for Security Patch
21. Scenario 20 — Full DR: Rebuild PG From Backup
22. Quick Reference Command Cheat Sheet
23. Escalation Matrix

1. CLUSTER ENVIRONMENT OVERVIEW

This runbook covers two production Kubernetes clusters running the Percona Operator for PostgreSQL. Both clusters were analyzed via `kubectl get -A all` and form the basis for all 20 incident scenarios documented herein.

Cluster-A

	Value
Control Plane	3 nodes (blt6v, c2jfb, k5zhn)
Total Nodes	6 (3 CP + 3 workers) — inferred from DaemonSet counts
CNI	Cilium — 6 agent pods + 2 operator replicas
PostgreSQL Cluster	pgkubcluster: i1-rvcz (5/5 Running), i2-d6hm (4/5 Running)
PgBouncer	2 replicas Running — b7f656889-k4j7c, b7f656889-s62tn
Backups	CronJob @ 50 23 * * * — 6 completed, 3 FAILED
Monitoring	kube-prometheus-stack (16d), Grafana NodePort :31000, PMM
Key Issue	i2-d6hm at 4/5 Ready 3 pgbackupmonitor failures (17–19d ago)

Cluster-B

	Value
Control Plane	5 nodes (6p4tj, 8m9sf, qp7zx, wqt29, wr4gw)
Total Nodes	10 (5 CP + 5 workers) — inferred from DaemonSet counts
CNI	Cilium — 10 agent pods + 2 operator replicas
PostgreSQL Cluster	salt-control-postgres: i1-hpkf (5/5), i2-5tf6 (5/5), i3-ptk4 (5/5)
PgBouncer	Deployment 0/0 Ready — DEGRADED (ReplicaSet 0 desired)
Backups	CronJob @ 50 23 * * * — 13 jobs ALL Complete (~5m each)
Monitoring	kube-prometheus-stack (90d), Grafana, PMM NodePort :31968
Key Issue	PgBouncer 0/0 controller-mgr 5–11 restarts scheduler 4–10 restarts

SCENARIO 1: Worker Node Kernel Panic & PostgreSQL Primary Failover

Cluster	Cluster-A	Severity	P1 — Critical
Operations	DRAIN NODE RESCHEDULE	Impact	PG primary on affected node; data unavailability risk

Context: Worker hosting pgkubcluster-i1-rvcz-0 (5/5) suffers kernel panic. Node goes NotReady. Patroni must orchestrate automatic failover to replica i2-d6hm-0. PgBouncer re-routes connections.

Pre-Remediation Backups

#	Backup Action	Command
1	Snapshot etcd	<code>ETCDCTL_API=3 etcdctl snapshot save /backup/etcd-pre.db</code>
2	Backup PG cluster CR	<code>kubectl -n postgres-operator get perconapgcluster -o yaml > /backup/pg-cr.yaml</code>
3	Backup all namespace state	<code>kubectl get all -A -o yaml > /backup/cluster-state.yaml</code>
4	Verify pgBackRest backup	<code>kubectl -n postgres-operator exec <repo-host> -c pgbackrest -- pgbackrest info</code>
5	Trigger fresh PG backup	<code>kubectl -n postgres-operator create job --from=cronjob/<pgbackupmonitor> pre-maint-\$(date +%s)</code>

Remediation Steps

#	Action	Command / Detail
1	Confirm node status	<code>kubectl get nodes -o wide grep NotReady</code>
2	Check PG cluster state	<code>kubectl -n postgres-operator get perconapgcluster pgkubcluster -o yaml grep -A5 status</code>
3	Verify Patroni failover	<code>kubectl -n postgres-operator exec pgkubcluster-i2-d6hm-0 -c database -- patronictl list</code>
4	If no auto-failover	<code>patronictl failover --leader i1-rvcz --candidate i2-d6hm --force</code>
5	Cordon bad node	<code>kubectl cordon <node-name></code>
6	Drain node safely	<code>kubectl drain <node> --ignore-daemonsets --delete-emptydir-data --grace-period=60 --timeout=300s</code>
7	Force-delete stuck pod	<code>kubectl -n postgres-operator delete pod pgkubcluster-i1-rvcz-0 --force --grace-period=0</code>
8	Verify new primary	<code>psql -c 'SELECT pg_is_in_recovery();'</code>
9	Replace/reboot node	<code>ssh <node> sudo reboot</code>

10	Uncordon when healthy	<code>kubectl uncordon <node-name></code>
----	-----------------------	---

Post-Remediation Verification

#	Verification	Command	Expected
1	All nodes Ready	<code>kubectl get nodes</code>	All STATUS=Ready
2	All pods Running	<code>kubectl get pods -A grep -v Running grep -v Completed</code>	No output
3	PG primary writable	<code>psql -c 'SELECT pg_is_in_recovery();'</code>	false
4	Replication streaming	<code>psql -c 'SELECT state FROM pg_stat_replication;'</code>	streaming
5	PgBouncer connected	<code>SHOW POOLS;</code>	Active > 0
6	Backup healthy	<code>pgbackrest info</code>	Recent backup
7	Services up	<code>kubectl -n postgres-operator get svc</code>	IPs assigned
8	DaemonSets complete	<code>kubectl get ds -A</code>	DESIRED = READY

WARNING: Patroni uses 30s failover timeout. Force-deleting the pod releases the leader lock. Always verify writes.

RESOLVED: Primary failover verified. Writes confirmed on new primary. PgBouncer re-routed.

SCENARIO 2: etcd Disk Pressure Forces Control Plane Node Replacement

Cluster	Cluster-A (3 CP)	Severity	P1 — Critical
Operations	REPLACE NODE DRAIN NODE	Impact	etcd quorum at risk; API server latency spikes

Context: Control plane node blt6v reports DiskPressure. etcd shows high fsync latency. With 3-node quorum, losing 1 member maintains quorum (2/3) but severely degrades performance.

Pre-Remediation Backups

#	Backup Action	Command
1	Snapshot etcd	<code>ETCDCTL_API=3 etcdctl snapshot save /backup/etcd-pre.db</code>
2	Backup PG cluster CR	<code>kubectl -n postgres-operator get perconapgcluster -o yaml > /backup/pg-cr.yaml</code>
3	Backup all namespace state	<code>kubectl get all -A -o yaml > /backup/cluster-state.yaml</code>
4	Verify pgBackRest backup	<code>kubectl -n postgres-operator exec <repo-host> -c pgbackrest -- pgbackrest info</code>
5	Trigger fresh PG backup	<code>kubectl -n postgres-operator create job --from=cronjob/<pgbackupmonitor> pre-maint-\$(date +%s)</code>
6	Backup kubeadm config	<code>kubectl -n kube-system get cm kubeadm-config -o yaml > /backup/kubeadm.yaml</code>

Remediation Steps

#	Action	Command / Detail
1	Check etcd health	<code>etcdctl endpoint health --cluster</code>
2	Check disk usage	<code>ssh blt6v 'df -h /var/lib/etcd && du -sh /var/lib/etcd'</code>
3	Defragment if possible	<code>etcdctl defrag --endpoints=https://[blt6v]:2379</code>
4	If HW failing, provision	Use IaC to provision new CP node with identical role
5	Cordon old CP node	<code>kubectl cordon ...-blt6v</code>
6	Drain non-critical pods	<code>kubectl drain ...-blt6v --ignore-daemonsets --delete-emptydir-data</code>
7	Remove etcd member	<code>etcdctl member remove <member-id></code>
8	Join new CP to cluster	<code>kubeadm join --control-plane</code>
9	Verify etcd quorum	<code>etcdctl endpoint status --cluster --write-out=table</code>
10	Delete old node object	<code>kubectl delete node ...-blt6v</code>

Post-Remediation Verification

#	Verification	Command	Expected
1	All nodes Ready	<code>kubectl get nodes</code>	<i>All Ready</i>
2	Pods healthy	<code>kubectl get pods -A grep -v Running grep -v Completed</code>	<i>Clean</i>
3	DaemonSets	<code>kubectl get ds -A</code>	<i>DESIRED = READY</i>
4	Services	<code>kubectl get svc -A</code>	<i>IPs assigned</i>
5	Events clean	<code>kubectl get events -A --sort-by=.lastTimestamp tail</code>	<i>No errors</i>
6	etcd quorum	<code>etcdctl endpoint health --cluster</code>	<i>All healthy</i>
7	PG unaffected	<code>patronictl list</code>	<i>Leader + replica intact</i>

WARNING: Cluster-A: 3 CPs = max 1 etcd failure tolerated. Cluster-B: 5 CPs = max 2. Always replace before removing another.

RESOLVED: New CP joined. etcd quorum restored at 3/3. API latency normalized.

SCENARIO 3: PostgreSQL Replica OOMKilled — Resource Resize Required

Cluster	Cluster-A	Severity	P2 — High
Operations	RESIZE POD	Impact	i2-d6hm-0 at 4/5 Ready; sidecar container OOMKilled

Context: Observed: pgkubcluster-i2-d6hm-0 running 4/5 containers. Investigation reveals the pgbackrest sidecar was OOMKilled (Exit Code 137). Memory limit too low for backup workload.

Pre-Remediation Backups

#	Backup Action	Command
1	Backup PG cluster CR	<code>kubectl -n postgres-operator get perconapgcluster -o yaml > /backup/pg-cr.yaml</code>
2	Backup PgBouncer config	<code>kubectl -n postgres-operator get configmap -l app=pgbouncer -o yaml > /backup/pgb-cm.yaml</code>
3	Verify pgBackRest backup	<code>kubectl -n postgres-operator exec <repo-host> -c pgbackrest -- pgbackrest info</code>
4	Trigger fresh PG backup	<code>kubectl -n postgres-operator create job --from=cronjob/<pgbackupmonitor> pre-maint-\$(date +%s)</code>

Remediation Steps

#	Action	Command / Detail
1	Identify OOMKilled container	<code>kubectl -n postgres-operator describe pod pgkubcluster-i2-d6hm-0 grep -B2 OOMKilled</code>
2	Check current limits	<code>kubectl -n postgres-operator get perconapgcluster pgkubcluster -o yaml grep -A20 resources</code>
3	Review actual memory	<code>kubectl -n postgres-operator top pod pgkubcluster-i2-d6hm-0 --containers</code>
4	Prometheus history	Query: <code>container_memory_working_set_bytes{pod=~'pgkubcluster.*'}</code>
5	Patch CR to increase	<code>kubectl -n postgres-operator patch perconapgcluster pgkubcluster --type merge -p '{"spec":{"instances":[{"name":"i2","resources":{"limits":{"memory":"2Gi"}}}]}}'</code>
6	Operator rolls StatefulSet	<code>kubectl -n postgres-operator get pods -w</code>
7	Verify 5/5 containers	<code>kubectl -n postgres-operator get pod pgkubcluster-i2-d6hm-0</code>
8	Validate replication lag	<code>psql -c 'SELECT sent_lsn, replay_lsn FROM pg_stat_replication;'</code>

Post-Remediation Verification

#	Verification	Command	Expected
1	PG primary writable	<code>psql -c 'SELECT pg_is_in_recovery();'</code>	<i>false</i>
2	Replication streaming	<code>SELECT * FROM pg_stat_replication;</code>	<i>state=streaming</i>
3	Patroni cluster	<code>patronictl list</code>	<i>1 Leader + Replicas</i>
4	PgBouncer pools	<code>SHOW POOLS;</code>	<i>Active pool</i>
5	Backup health	<code>pgbackrest info</code>	<i>Within RPO</i>
6	PG services	<code>kubectl -n postgres-operator get svc grep postgres</code>	<i>primary + replicas</i>
7	Pod 5/5	<code>kubectl get pod i2-d6hm-0</code>	<i>5/5 Running</i>
8	Memory applied	<code>kubectl describe pod ... grep Limits</code>	<i>memory: 2Gi</i>

WARNING: CR patch triggers rolling restart. Always resize replicas FIRST, then primary to avoid double failover.

RESOLVED: Pod i2-d6hm-0 now 5/5 Ready. Memory raised to 2Gi. Replication lag: 0 bytes.

SCENARIO 4: Cilium Agent CrashLoop After Node OS Upgrade

Cluster	Cluster-B	Severity	P1 — Critical
Operations	DRAIN NODE RESCHEDULE	Impact	All pods on node lose networking; Cilium BPF incompatibility

Context: After OS kernel upgrade on a worker node in Cluster-B, the Cilium DaemonSet pod enters CrashLoopBackOff due to BPF incompatibility. All pods on that node lose connectivity.

Pre-Remediation Backups

#	Backup Action	Command
1	Snapshot etcd	<code>ETCDCTL_API=3 etcdctl snapshot save /backup/etcd-pre.db</code>
2	Backup PG cluster CR	<code>kubectl -n postgres-operator get perconapgcluster -o yaml > /backup/pg-cr.yaml</code>
3	Backup all namespace state	<code>kubectl get all -A -o yaml > /backup/cluster-state.yaml</code>
4	Verify pgBackRest backup	<code>kubectl -n postgres-operator exec <repo-host> -c pgbackrest -- pgbackrest info</code>
5	Trigger fresh PG backup	<code>kubectl -n postgres-operator create job --from=cronjob/<pgbackupmonitor> pre-maint-\$(date +%s)</code>

Remediation Steps

#	Action	Command / Detail
1	Identify crashing pod	<code>kubectl -n kube-system get pods -l app.kubernetes.io/name=cilium-agent -o wide grep CrashLoop</code>
2	Check Cilium logs	<code>kubectl -n kube-system logs <pod> --previous tail -50</code>
3	Cordon node	<code>kubectl cordon <node></code>
4	Drain workloads	<code>kubectl drain <node> --ignore-daemonsets --delete-emptydir-data</code>
5	Check if PG affected	<code>kubectl -n postgres-operator get pods -o wide grep <node></code>
6	Verify Patroni state	<code>patronictl list</code>
7	Fix kernel/BPF	<code>ssh <node> 'apt install linux-headers-\$(uname -r) && systemctl restart kubelet'</code>
8	Delete stuck Cilium pod	<code>kubectl -n kube-system delete pod <cilium-pod></code>
9	Verify Cilium health	<code>kubectl -n kube-system exec <cilium-pod> -- cilium status</code>
10	Uncordon node	<code>kubectl uncordon <node></code>

Post-Remediation Verification

#	Verification	Command	Expected
1	All nodes Ready	<code>kubectl get nodes</code>	<i>All STATUS=Ready</i>
2	All pods Running	<code>kubectl get pods -A grep -v Running grep -v Completed</code>	<i>No output</i>
3	PG primary writable	<code>psql -c 'SELECT pg_is_in_recovery();'</code>	<i>false</i>
4	Replication streaming	<code>psql -c 'SELECT state FROM pg_stat_replication;'</code>	<i>streaming</i>
5	PgBouncer connected	<code>SHOW POOLS;</code>	<i>Active > 0</i>
6	Backup healthy	<code>pgbackrest info</code>	<i>Recent backup</i>
7	Services up	<code>kubectl -n postgres-operator get svc</code>	<i>IPs assigned</i>
8	DaemonSets complete	<code>kubectl get ds -A</code>	<i>DESIRED = READY</i>

RESOLVED: Cilium recovered. DaemonSet 10/10 ready. Pod networking restored.

SCENARIO 5: PgBouncer Deployment Scaled to Zero (Cluster-B Observed)

Cluster	Cluster-B	Severity	P1 — Critical
Operations	RESCHEDULE RESIZE POD	Impact	All app DB connections severed; PgBouncer at 0/0 replicas

Context: Observed: deployment salt-control-postgres-pgbouncer = 0/0 Ready with ReplicaSet at 0 desired. Applications cannot connect to PostgreSQL through the connection pooler.

Pre-Remediation Backups

#	Backup Action	Command
1	Backup PG cluster CR	<code>kubectl -n postgres-operator get perconapgcluster -o yaml > /backup/pg-cr.yaml</code>
2	Backup PgBouncer config	<code>kubectl -n postgres-operator get configmap -l app=pgbouncer -o yaml > /backup/pgb-cm.yaml</code>
3	Verify pgBackRest backup	<code>kubectl -n postgres-operator exec <repo-host> -c pgbackrest -- pgbackrest info</code>
4	Trigger fresh PG backup	<code>kubectl -n postgres-operator create job --from=cronjob/<pgbackupmonitor> pre-maint-\$(date +%s)</code>

Remediation Steps

#	Action	Command / Detail
1	Check CR proxy spec	<code>kubectl -n postgres-operator get perconapgcluster salt-control-postgres -o yaml grep -A20 proxy</code>
2	Check operator logs	<code>kubectl -n postgres-operator logs deploy/percona-postgresql-operator grep pgbouncer</code>
3	Check resource quotas	<code>kubectl -n postgres-operator describe resourcequota</code>
4	Check events	<code>kubectl -n postgres-operator get events --sort-by=.lastTimestamp grep pgbouncer</code>
5	Fix: scale replicas in CR	<code>kubectl -n postgres-operator patch perconapgcluster salt-control-postgres --type merge -p '{"spec":{"proxy":{"pgBouncer":{"replicas":2}}}}'</code>
6	Watch pods come up	<code>kubectl -n postgres-operator get pods -w -l postgres-operator.crunchydata.com/role=pgbouncer</code>
7	Verify backend connect	<code>psql -h 127.0.0.1 -p 5432 -U pgbouncer pgbouncer -c 'SHOW SERVERS;'</code>
8	Verify app connectivity	<code>psql -h <pgbouncer-svc> -U <app-user> <db> -c 'SELECT 1;'</code>

Post-Remediation Verification

#	Verification	Command	Expected
1	PG primary writable	psql -c 'SELECT pg_is_in_recovery();'	false
2	Replication streaming	SELECT * FROM pg_stat_replication;	state=streaming
3	Patroni cluster	patronictl list	1 Leader + Replicas
4	PgBouncer pools	SHOW POOLS;	Active pool
5	Backup health	pgbackrest info	Within RPO
6	PG services	kubectl -n postgres-operator get svc grep postgres	primary + replicas

RESOLVED: PgBouncer scaled to 2 replicas. Application connectivity fully restored.

SCENARIO 6: CSI Volume Plugin Failure Blocks PVC Mounting

Cluster	Cluster-A	Severity	P1 — Critical
Operations	DRAIN NODE REPLACE NODE	Impact	PG pod stuck ContainerCreating; PVC cannot attach

Context: After node replacement, the PostgreSQL pod is scheduled to a node where the csi-ssccloud-node DaemonSet pod is failing. PVC cannot be attached.

Pre-Remediation Backups

#	Backup Action	Command
1	Snapshot etcd	<code>ETCDCTL_API=3 etcdctl snapshot save /backup/etcd-pre.db</code>
2	Backup PG cluster CR	<code>kubectl -n postgres-operator get perconapgcluster -o yaml > /backup/pg-cr.yaml</code>
3	Backup all namespace state	<code>kubectl get all -A -o yaml > /backup/cluster-state.yaml</code>
4	Verify pgBackRest backup	<code>kubectl -n postgres-operator exec <repo-host> -c pgbackrest -- pgbackrest info</code>
5	Trigger fresh PG backup	<code>kubectl -n postgres-operator create job --from=cronjob/<pgbackupmonitor> pre-maint-\$(date +%s)</code>

Remediation Steps

#	Action	Command / Detail
1	Check stuck pod events	<code>kubectl -n postgres-operator describe pod <pg-pod> grep -A10 Events</code>
2	Check CSI node pods	<code>kubectl -n kube-system get pods -l app=csi-ssccloud-node -o wide</code>
3	Check VolumeAttachment	<code>kubectl get volumeattachment grep <pv-name></code>
4	Cordon bad node	<code>kubectl cordon <node></code>
5	Delete stuck PG pod	<code>kubectl -n postgres-operator delete pod <pg-pod></code>
6	If PVC stuck, detach	<code>kubectl delete volumeattachment <va-name></code>
7	Verify rescheduled	<code>kubectl -n postgres-operator get pod <pg-pod> -o wide</code>
8	Fix CSI on original	<code>kubectl -n kube-system delete pod <csi-node-pod></code>
9	Uncordon when healthy	<code>kubectl uncordon <node></code>

Post-Remediation Verification

#	Verification	Command	Expected
---	--------------	---------	----------

1	All nodes Ready	<code>kubectl get nodes</code>	<i>All STATUS=Ready</i>
2	All pods Running	<code>kubectl get pods -A grep -v Running grep -v Completed</code>	<i>No output</i>
3	PG primary writable	<code>psql -c 'SELECT pg_is_in_recovery();'</code>	<i>false</i>
4	Replication streaming	<code>psql -c 'SELECT state FROM pg_stat_replication;'</code>	<i>streaming</i>
5	PgBouncer connected	<code>SHOW POOLS;</code>	<i>Active > 0</i>
6	Backup healthy	<code>pgbackrest info</code>	<i>Recent backup</i>
7	Services up	<code>kubectl -n postgres-operator get svc</code>	<i>IPs assigned</i>
8	DaemonSets complete	<code>kubectl get ds -A</code>	<i>DESIRED = READY</i>

WARNING: Never force-delete a PG pod with PVC in use. Can cause data corruption if volume still attached.

RESOLVED: PVC mounted on healthy node. PostgreSQL pod running. CSI fixed on original node.

SCENARIO 7: pgbackupmonitor CronJob Failures (Cluster-A Observed)

Cluster	Cluster-A	Severity	P2 — High
Operations	RESCHEDULE	Impact	3 consecutive backup monitor failures; backup integrity at risk

Context: Observed: jobs 29518550 (19d), 29519990 (18d), 29521430 (17d) all Failed 0/1. Subsequent runs succeeded. Root cause and alerting needed.

Pre-Remediation Backups

#	Backup Action	Command
1	Backup PG cluster CR	<code>kubectl -n postgres-operator get perconapgcluster -o yaml > /backup/pg-cr.yaml</code>
2	Backup PgBouncer config	<code>kubectl -n postgres-operator get configmap -l app=pgbouncer -o yaml > /backup/pgb-cm.yaml</code>
3	Verify pgBackRest backup	<code>kubectl -n postgres-operator exec <repo-host> -c pgbackrest -- pgbackrest info</code>
4	Trigger fresh PG backup	<code>kubectl -n postgres-operator create job --from=cronjob/<pgbackupmonitor> pre-maint-\$(date +%s)</code>

Remediation Steps

#	Action	Command / Detail
1	List all job runs	<code>kubectl -n postgres-operator get jobs -l postgres-operator.crunchydata.com/pgbackrest</code>
2	Check failed job logs	<code>kubectl -n postgres-operator logs job/pgkubcluster-pgbackupmonitor-29518550</code>
3	Check repo-host	<code>kubectl -n postgres-operator get pod pgkubcluster-repo-host-0</code>
4	Check pgBackRest info	<code>kubectl -n postgres-operator exec pgkubcluster-repo-host-0 -c pgbackrest -- pgbackrest info</code>
5	Check node events	<code>kubectl get events -A --field-selector reason=NodeNotReady</code>
6	Clean up failed jobs	<code>kubectl -n postgres-operator delete job pgkubcluster-pgbackupmonitor-{29518550,29519990,29521430}</code>
7	Trigger manual check	<code>kubectl -n postgres-operator create job --from=cronjob/pgkubcluster-pgbackupmonitor manual-\$(date +%s)</code>
8	Set up failure alert	PrometheusRule: kube_job_status_failed > 0 for pgbackupmonitor

Post-Remediation Verification

#	Verification	Command	Expected
1	Manual job passed	<code>kubectl get jobs grep manual</code>	<i>Complete 1/1</i>
2	Repo-host healthy	<code>kubectl get pod repo-host-0</code>	<i>2/2 Running</i>
3	Backup integrity	<code>pgbackrest info</code>	<i>Recent backup</i>
4	CronJob active	<code>kubectl get cronjob</code>	<i>LAST SCHEDULE recent</i>
5	PG primary writable	<code>psql -c 'SELECT pg_is_in_recovery();'</code>	<i>false</i>
6	Replication streaming	<code>SELECT * FROM pg_stat_replication;</code>	<i>state=streaming</i>
7	Patroni cluster	<code>patronictl list</code>	<i>1 Leader + Replicas</i>
8	PgBouncer pools	<code>SHOW POOLS;</code>	<i>Active pool</i>
9	Backup health	<code>pgbackrest info</code>	<i>Within RPO</i>
10	PG services	<code>kubectl -n postgres-operator get svc grep postgres</code>	<i>primary + replicas</i>

RESOLVED: Root cause: node maintenance during backup window. Alert configured for consecutive failures.

SCENARIO 8: Control Plane Certificate Expiry

Cluster	Cluster-B (5 CP)	Severity	P1 — Critical
Operations	REPLACE NODE	Impact	API server on CP unreachable; scheduler shows 9 restarts

Context: Cluster-B node qp7zx kube-apiserver TLS certificates expired. kube-scheduler has 9 restarts (observed) from repeated local API connection failures.

Pre-Remediation Backups

#	Backup Action	Command
1	Snapshot etcd	<code>ETCDCTL_API=3 etcdctl snapshot save /backup/etcd-pre.db</code>
2	Backup PG cluster CR	<code>kubectl -n postgres-operator get perconapgcluster -o yaml > /backup/pg-cr.yaml</code>
3	Backup all namespace state	<code>kubectl get all -A -o yaml > /backup/cluster-state.yaml</code>
4	Verify pgBackRest backup	<code>kubectl -n postgres-operator exec <repo-host> -c pgbackrest -- pgbackrest info</code>
5	Trigger fresh PG backup	<code>kubectl -n postgres-operator create job --from=cronjob/<pgbackupmonitor> pre-maint-\$(date +%s)</code>

Remediation Steps

#	Action	Command / Detail
1	Check cert expiry	<code>ssh qp7zx 'kubeadm certs check-expiration'</code>
2	Check API server logs	<code>ssh qp7zx 'crictl logs \$(crictl ps -a --name kube-apiserver -q) tail -20'</code>
3	Renew certificates	<code>ssh qp7zx 'kubeadm certs renew all'</code>
4	Restart static pods	<code>ssh qp7zx 'mv /etc/kubernetes/manifests/*.yaml /tmp/ && sleep 10 && mv /tmp/*.yaml /etc/kubernetes/manifests/'</code>
5	Verify API responds	<code>kubectl --server=https://<qp7zx-ip>:6443 get nodes</code>
6	If renewal fails	Remove from LB, drain, remove etcd member, provision new CP
7	Set up cert monitoring	Alert at 30 days before expiry via Prometheus

Post-Remediation Verification

#	Verification	Command	Expected
1	All nodes Ready	<code>kubectl get nodes</code>	All Ready

2	Pods healthy	<code>kubectl get pods -A grep -v Running grep -v Completed</code>	<i>Clean</i>
3	DaemonSets	<code>kubectl get ds -A</code>	<i>DESIRED = READY</i>
4	Services	<code>kubectl get svc -A</code>	<i>IPs assigned</i>
5	Events clean	<code>kubectl get events -A --sort-by=.lastTimestamp tail</code>	<i>No errors</i>
6	Cert expiry	<code>kubeadm certs check-expiration</code>	<i>All > 30d</i>
7	Scheduler stable	<code>kubectl get pods -n kube-system grep scheduler</code>	<i>0 new restarts</i>

WARNING: 5 CP = tolerate 2 etcd failures. Cert expiry can cascade across nodes. Implement automated rotation.

RESOLVED: Certificates renewed. API server restored. Scheduler restarts stabilized.

SCENARIO 9: PostgreSQL Instance i2-d6hm 4/5 Not Ready (Cluster-A)

Cluster	Cluster-A	Severity	P2 — High
Operations	RESIZE POD RESCHEDULE	Impact	Replica degraded; failover target not fully healthy

Context: Direct observation: i2-d6hm-0 = 4/5 Ready. StatefulSet 0/1 Ready. This is the failover target for primary i1-rvcz. The 5 containers: database, pgbackrest, pgbackrest-config, pgbadger, replication-cert-copy.

Pre-Remediation Backups

#	Backup Action	Command
1	Backup PG cluster CR	<code>kubectl -n postgres-operator get perconapgcluster -o yaml > /backup/pg-cr.yaml</code>
2	Backup PgBouncer config	<code>kubectl -n postgres-operator get configmap -l app=pgbouncer -o yaml > /backup/pgb-cm.yaml</code>
3	Verify pgBackRest backup	<code>kubectl -n postgres-operator exec <repo-host> -c pgbackrest -- pgbackrest info</code>
4	Trigger fresh PG backup	<code>kubectl -n postgres-operator create job --from=cronjob/<pgbackupmonitor> pre-maint-\$(date +%s)</code>

Remediation Steps

#	Action	Command / Detail
1	Find not-ready container	<code>kubectl -n postgres-operator get pod i2-d6hm-0 -o jsonpath='{.status.containerStatuses[*].name}{.status.containerStatuses[*].ready}'</code>
2	Check events	<code>kubectl -n postgres-operator describe pod i2-d6hm-0 tail -30</code>
3	Check container logs	<code>kubectl -n postgres-operator logs i2-d6hm-0 -c <failing-container> --previous</code>
4	If OOMKilled: resize	<code>kubectl patch perconapgcluster pgkubcluster --type merge -p '{"spec":{"instances":[{"name":"i2","resources":{"limits":{"memory":"2Gi"}}}]}}'</code>
5	If readiness probe fail	<code>kubectl get pod i2-d6hm-0 -o yaml grep -A10 readinessProbe</code>
6	If replication broken	<code>kubectl annotate perconapgcluster pgkubcluster --overwrite postgres-operator.crunchydata.com/pgbackrest-restore=i2-d6hm</code>
7	Force pod restart	<code>kubectl -n postgres-operator delete pod i2-d6hm-0</code>
8	Verify 5/5	<code>kubectl -n postgres-operator get pod i2-d6hm-0</code>

Post-Remediation Verification

#	Verification	Command	Expected
1	PG primary writable	psql -c 'SELECT pg_is_in_recovery();'	false
2	Replication streaming	SELECT * FROM pg_stat_replication;	state=streaming
3	Patroni cluster	patronictl list	1 Leader + Replicas
4	PgBouncer pools	SHOW POOLS;	Active pool
5	Backup health	pgbackrest info	Within RPO
6	PG services	kubectl -n postgres-operator get svc grep postgres	primary + replicas

RESOLVED: All 5 containers Ready. Replication lag zero. Failover target healthy.

SCENARIO 10: Rolling Kubernetes Version Upgrade Across 10 Nodes

Cluster	Cluster-B	Severity	P3 — Planned
Operations	DRAIN NODE REPLACE NODE	Impact	Sequential drain/upgrade; PG must remain available

Context: Cluster-B: 5 CPs + 5 workers = 10 nodes. K8s version upgrade requires draining each node sequentially. salt-control-postgres (3 instances) must maintain availability throughout.

Pre-Remediation Backups

#	Backup Action	Command
1	Snapshot etcd	<code>ETCDCTL_API=3 etcdctl snapshot save /backup/etcd-pre.db</code>
2	Backup PG cluster CR	<code>kubectl -n postgres-operator get perconapgcluster -o yaml > /backup/pg-cr.yaml</code>
3	Backup all namespace state	<code>kubectl get all -A -o yaml > /backup/cluster-state.yaml</code>
4	Verify pgBackRest backup	<code>kubectl -n postgres-operator exec <repo-host> -c pgbackrest -- pgbackrest info</code>
5	Trigger fresh PG backup	<code>kubectl -n postgres-operator create job --from=cronjob/<pgbackupmonitor> pre-maint-\$(date +%s)</code>

Remediation Steps

#	Action	Command / Detail
1	Pre-check PG health	<code>kubectl -n postgres-operator exec salt-control-postgres-i1-hpkf-0 -c database -- patronictl list</code>
2	Pre-check backups	<code>kubectl -n postgres-operator exec salt-control-postgres-repo-host-0 -c pgbackrest -- pgbackrest info</code>
3	Plan upgrade order	Workers first (avoid PG primary), then CPs one at a time
4	Cordon worker 1	<code>kubectl cordon <worker-1></code>
5	Drain worker 1	<code>kubectl drain <worker-1> --ignore-daemonsets --delete-emptydir-data --grace-period=120</code>
6	Upgrade kubelet/kubeadm	<code>ssh <worker-1> 'apt install kubelet=<ver> kubeadm=<ver> && systemctl restart kubelet'</code>
7	Uncordon worker 1	<code>kubectl uncordon <worker-1></code>
8	Repeat for all workers	Iterate steps 4–7 for each remaining worker node
9	Upgrade CPs one at a time	<code>kubeadm upgrade apply <ver> on first CP</code> <code>kubeadm upgrade node on remaining CPs</code>

10	Final health check	<code>kubectl get nodes && patronictl list</code>
----	--------------------	---

Post-Remediation Verification

#	Verification	Command	Expected
1	All nodes Ready	<code>kubectl get nodes</code>	All STATUS=Ready
2	All pods Running	<code>kubectl get pods -A grep -v Running grep -v Completed</code>	No output
3	PG primary writable	<code>psql -c 'SELECT pg_is_in_recovery();'</code>	false
4	Replication streaming	<code>psql -c 'SELECT state FROM pg_stat_replication;'</code>	streaming
5	PgBouncer connected	<code>SHOW POOLS;</code>	Active > 0
6	Backup healthy	<code>pgbackrest info</code>	Recent backup
7	Services up	<code>kubectl -n postgres-operator get svc</code>	IPs assigned
8	DaemonSets complete	<code>kubectl get ds -A</code>	DESIRED = READY
9	K8s version uniform	<code>kubectl get nodes -o wide</code>	All showing new version

WARNING: Never drain PG primary node without confirming replica is 5/5 with zero replication lag.

RESOLVED: All 10 nodes upgraded. PostgreSQL maintained 100% availability via rolling drain.

SCENARIO 11: Prometheus Stack OOMKilled During Metric Spike

Cluster	Both Clusters	Severity	P2 — High
Operations	RESIZE POD	Impact	Monitoring blackout; no alerting for PG issues

Context: prometheus-kube-prometheus-stack-prometheus-0 OOMKilled during metric cardinality spike. Cluster-A shows 3 restarts (9d ago) confirming previous OOM events.

Pre-Remediation Backups

#	Backup Action	Command
1	Backup Prometheus CR	<code>kubectl -n monitoring get prometheus -o yaml > /backup/prom-cr.yaml</code>
2	Backup alerting rules	<code>kubectl -n monitoring get prometheusrule -o yaml > /backup/rules.yaml</code>
3	Backup dashboards	<code>kubectl -n monitoring get cm -l grafana_dashboard=1 -o yaml > /backup/dash.yaml</code>

Remediation Steps

#	Action	Command / Detail
1	Check pod status	<code>kubectl -n monitoring describe pod prometheus-kube-prometheus-stack-prometheus-0 grep -A5 'Last State'</code>
2	Check resources	<code>kubectl -n monitoring get sts prometheus-kube-prometheus-stack-prometheus -o yaml grep -A10 resources</code>
3	Find cardinality spike	Query: <code>count({__name__=~'.+'}) by (__name__) topk(20)</code>
4	Increase memory	<code>kubectl -n monitoring edit prometheus kube-prometheus-stack-prometheus</code> Set <code>spec.resources.limits.memory: 4Gi</code>
5	Add retention limits	<code>spec.retention: 15d spec.retentionSize: 40GB</code>
6	Restart Prometheus	<code>kubectl -n monitoring delete pod prometheus-kube-prometheus-stack-prometheus-0</code>
7	Verify targets	<code>curl -s http://<prom-svc>:9090/api/v1/targets jq '.data.activeTargets length'</code>

Post-Remediation Verification

#	Verification	Command	Expected
---	--------------	---------	----------

1	Prometheus stable	<code>kubectl get pod prometheus-...-0 -n monitoring</code>	2/2 Running, 0 restarts
2	Memory OK	<code>kubectl top pod prometheus-...-0 -n monitoring</code>	Below 4Gi
3	Alertmanager OK	<code>curl <am-svc>:9093/api/v2/status</code>	ready
4	PG metrics	Query: <code>pg_up</code>	1 for all instances

RESOLVED: Memory increased to 4Gi. Retention configured. No further OOM events.

SCENARIO 12: PVC Storage Full on PG Primary (WAL Accumulation)

Cluster	Cluster-A	Severity	P1 — Critical
Operations	RESIZE POD	Impact	PG stops writes: PANIC could not write WAL

Context: Primary i1-rvcz-0 fills its PVC because WAL segments accumulate — replica i2-d6hm (4/5 state) is not consuming WAL from its inactive replication slot.

Pre-Remediation Backups

#	Backup Action	Command
1	Backup PG cluster CR	<code>kubectl -n postgres-operator get perconapgcluster -o yaml > /backup/pg-cr.yaml</code>
2	Backup PgBouncer config	<code>kubectl -n postgres-operator get configmap -l app=pgbouncer -o yaml > /backup/pgb-cm.yaml</code>
3	Verify pgBackRest backup	<code>kubectl -n postgres-operator exec <repo-host> -c pgbackrest -- pgbackrest info</code>
4	Trigger fresh PG backup	<code>kubectl -n postgres-operator create job --from=cronjob/<pgbackupmonitor> pre-maint-\$(date +%s)</code>

Remediation Steps

#	Action	Command / Detail
1	Check disk usage	<code>kubectl -n postgres-operator exec pgkubcluster-i1-rvcz-0 -c database -- df -h /pgdata</code>
2	Check WAL directory	<code>kubectl -n postgres-operator exec pgkubcluster-i1-rvcz-0 -c database -- du -sh /pgdata/pg16/pg_wal</code>
3	Check replication slots	<code>psql -c "SELECT slot_name, active, pg_wal_lsn_diff(pg_current_wal_lsn(), restart_lsn) FROM pg_replication_slots;"</code>
4	Emergency: drop slot	<code>psql -c "SELECT pg_drop_replication_slot('slot');" </code>
5	Check SC expansion	<code>kubectl get sc -o yaml grep allowVolumeExpansion</code>
6	Expand PVC via CR	<code>kubectl patch perconapgcluster pgkubcluster --type merge -p '{"spec":{"instances":[{"name":"i1", "dataVolumeClaimSpec":{"resources":{"requests":{"storage":"100Gi"}}}}]}}'</code>
7	Verify PVC resized	<code>kubectl -n postgres-operator get pvc grep i1-rvcz</code>
8	Fix replica (Scenario 9)	Resolve i2-d6hm to consume WAL

Post-Remediation Verification

#	Verification	Command	Expected
1	PG primary writable	<code>psql -c 'SELECT pg_is_in_recovery();'</code>	<i>false</i>
2	Replication streaming	<code>SELECT * FROM pg_stat_replication;</code>	<i>state=streaming</i>
3	Patroni cluster	<code>patronictl list</code>	<i>1 Leader + Replicas</i>
4	PgBouncer pools	<code>SHOW POOLS;</code>	<i>Active pool</i>
5	Backup health	<code>pgbackrest info</code>	<i>Within RPO</i>
6	PG services	<code>kubectl -n postgres-operator get svc grep postgres</code>	<i>primary + replicas</i>
7	PVC size	<code>kubectl get pvc grep i1-rvcz</code>	<i>100Gi Bound</i>
8	WAL stable	<code>du -sh pg_wal</code>	<i>Decreasing</i>

ALERT: NEVER delete WAL files from filesystem. Use `pg_drop_replication_slot()` or fix the replica.

RESOLVED: PVC expanded. Inactive slot cleared. Replica resynced. WAL directory stabilized.

SCENARIO 13: Node NotReady — Kubelet Memory Leak

Cluster	Cluster-B	Severity	P1 — Critical
Operations	DRAIN NODE REPLACE NODE	Impact	Node unresponsive; PG replica pods in Unknown state

Context: A worker node becomes NotReady from kubelet memory leak. kube-proxy pods show restarts indicating instability.

Pre-Remediation Backups

#	Backup Action	Command
1	Snapshot etcd	<code>ETCDCTL_API=3 etcdctl snapshot save /backup/etcd-pre.db</code>
2	Backup PG cluster CR	<code>kubectl -n postgres-operator get perconapgcluster -o yaml > /backup/pg-cr.yaml</code>
3	Backup all namespace state	<code>kubectl get all -A -o yaml > /backup/cluster-state.yaml</code>
4	Verify pgBackRest backup	<code>kubectl -n postgres-operator exec <repo-host> -c pgbackrest -- pgbackrest info</code>
5	Trigger fresh PG backup	<code>kubectl -n postgres-operator create job --from=cronjob/<pgbackupmonitor> pre-maint-\$(date +%s)</code>

Remediation Steps

#	Action	Command / Detail
1	Identify NotReady node	<code>kubectl get nodes grep NotReady</code>
2	Check kubelet	<code>ssh <node> 'systemctl status kubelet && journalctl -u kubelet --since "1h ago" tail -50'</code>
3	Check memory	<code>ssh <node> 'free -h && ps aux --sort=-%mem head'</code>
4	Cordon	<code>kubectl cordon <node></code>
5	Drain	<code>kubectl drain <node> --ignore-daemonsets --force --timeout=120s</code>
6	Force-delete Unknown pods	<code>kubectl get pods -A --field-selector spec.nodeName=<node> grep Unknown xargs kubectl delete --force</code>
7	Restart kubelet	<code>ssh <node> 'systemctl restart kubelet'</code>
8	If insufficient: replace	Provision new node, join cluster, delete old node
9	Verify PG rescheduled	<code>kubectl -n postgres-operator get pods -o wide</code>

Post-Remediation Verification

#	Verification	Command	Expected
1	All nodes Ready	<code>kubectl get nodes</code>	<i>All STATUS=Ready</i>
2	All pods Running	<code>kubectl get pods -A grep -v Running grep -v Completed</code>	<i>No output</i>
3	PG primary writable	<code>psql -c 'SELECT pg_is_in_recovery();'</code>	<i>false</i>
4	Replication streaming	<code>psql -c 'SELECT state FROM pg_stat_replication;'</code>	<i>streaming</i>
5	PgBouncer connected	<code>SHOW POOLS;</code>	<i>Active > 0</i>
6	Backup healthy	<code>pgbackrest info</code>	<i>Recent backup</i>
7	Services up	<code>kubectl -n postgres-operator get svc</code>	<i>IPs assigned</i>
8	DaemonSets complete	<code>kubectl get ds -A</code>	<i>DESIRED = READY</i>

RESOLVED: Node replaced. Kubelet upgraded. PG replica rescheduled and replication caught up.

SCENARIO 14: PG Operator CrashLoop After CRD Update

Cluster	Cluster-A	Severity	P1 — Critical
Operations	RESCHEDULE	Impact	Operator cannot reconcile; PG cluster unmanaged

Context: After CRD upgrade, operator enters CrashLoop due to schema validation errors between existing CR fields and new CRD schema.

Pre-Remediation Backups

#	Backup Action	Command
1	Backup ALL PG CRs	<code>kubectl get perconapgcluster -A -o yaml > /backup/all-pg-crs.yaml</code>
2	Backup CRD definition	<code>kubectl get crd perconapgclusters.pgv2.percona.com -o yaml > /backup/crd.yaml</code>
3	Backup operator deploy	<code>kubectl -n postgres-operator get deploy percona-postgresql-operator -o yaml > /backup/operator.yaml</code>
4	Verify PG backup exists	<code>kubectl -n postgres-operator exec <repo-host> -c pgbackrest -- pgbackrest info</code>

Remediation Steps

#	Action	Command / Detail
1	Check operator logs	<code>kubectl -n postgres-operator logs deploy/percona-postgresql-operator --previous tail -50</code>
2	Check CRD version	<code>kubectl get crd perconapgclusters.pgv2.percona.com -o yaml grep 'version:'</code>
3	Diff CR against schema	<code>kubectl explain perconapgcluster.spec --recursive</code>
4	Fix deprecated fields	<code>kubectl -n postgres-operator edit perconapgcluster pgkubcluster</code>
5	Restart operator	<code>kubectl -n postgres-operator rollout restart deploy/percona-postgresql-operator</code>
6	Watch reconciliation	<code>kubectl -n postgres-operator logs deploy/percona-postgresql-operator -f</code>

Post-Remediation Verification

#	Verification	Command	Expected
1	PG primary writable	<code>psql -c 'SELECT pg_is_in_recovery();'</code>	<i>false</i>

2	Replication streaming	<code>SELECT * FROM pg_stat_replication;</code>	<code>state=streaming</code>
3	Patroni cluster	<code>patronictl list</code>	<code>1 Leader + Replicas</code>
4	PgBouncer pools	<code>SHOW POOLS;</code>	<code>Active pool</code>
5	Backup health	<code>pgbackrest info</code>	<code>Within RPO</code>
6	PG services	<code>kubectl -n postgres-operator get svc grep postgres</code>	<code>primary + replicas</code>
7	Operator stable	<code>kubectl get deploy percona-postgresql-operator</code>	<code>READY 1/1</code>

WARNING: Always backup CRs before CRD upgrades: `kubectl get perconapgcluster -A -o yaml > backup.yaml`

RESOLVED: Operator reconciling. CR schema fixed. PG cluster fully managed.

SCENARIO 15: Scheduled Hardware Decommission (Memory DIMM Failure)

Cluster	Cluster-A	Severity	P2 — Planned
Operations	DRAIN NODE REPLACE NODE	Impact	Worker with ECC errors; hosts PgBouncer (s62tn, 12 restarts)

Context: ECC memory errors detected on node hosting pgbouncer-b7f656889-s62tn (12 restarts). Graceful decommission required with zero-downtime PgBouncer failover.

Pre-Remediation Backups

#	Backup Action	Command
1	Snapshot etcd	<code>ETCDCTL_API=3 etcdctl snapshot save /backup/etcd-pre.db</code>
2	Backup PG cluster CR	<code>kubectl -n postgres-operator get perconapgcluster -o yaml > /backup/pg-cr.yaml</code>
3	Backup all namespace state	<code>kubectl get all -A -o yaml > /backup/cluster-state.yaml</code>
4	Verify pgBackRest backup	<code>kubectl -n postgres-operator exec <repo-host> -c pgbackrest -- pgbackrest info</code>
5	Trigger fresh PG backup	<code>kubectl -n postgres-operator create job --from=cronjob/<pgbackupmonitor> pre-maint-\$(date +%s)</code>

Remediation Steps

#	Action	Command / Detail
1	Notify stakeholders	Send maintenance window notification
2	Verify PgBouncer HA	<code>kubectl -n postgres-operator get pods -o wide grep pgbouncer</code> # Confirm 2 replicas on different nodes
3	Cordon target	<code>kubectl cordon <node></code>
4	Drain gracefully	<code>kubectl drain <node> --ignore-daemonsets --grace-period=120</code>
5	Verify PgBouncer rescheduled	<code>kubectl -n postgres-operator get pods -o wide grep pgbouncer</code>
6	Provision replacement	IaC: Terraform/Ansible for new worker node
7	Join new node	<code>kubeadm join <control-plane-endpoint></code>
8	Label new node	<code>kubectl label node <new> node-role.kubernetes.io/worker=''</code>
9	Delete old node object	<code>kubectl delete node <old></code>

Post-Remediation Verification

#	Verification	Command	Expected
1	All nodes Ready	<code>kubectl get nodes</code>	<i>All STATUS=Ready</i>
2	All pods Running	<code>kubectl get pods -A grep -v Running grep -v Completed</code>	<i>No output</i>
3	PG primary writable	<code>psql -c 'SELECT pg_is_in_recovery();'</code>	<i>false</i>
4	Replication streaming	<code>psql -c 'SELECT state FROM pg_stat_replication;'</code>	<i>streaming</i>
5	PgBouncer connected	<code>SHOW POOLS;</code>	<i>Active > 0</i>
6	Backup healthy	<code>pgbackrest info</code>	<i>Recent backup</i>
7	Services up	<code>kubectl -n postgres-operator get svc</code>	<i>IPs assigned</i>
8	DaemonSets complete	<code>kubectl get ds -A</code>	<i>DESIRED = READY</i>

RESOLVED: Node replaced. PgBouncer maintained availability via second replica. Zero downtime.

SCENARIO 16: Network Partition Causes Split-Brain Risk

Cluster	Cluster-B	Severity	P1 — Critical
Operations	DRAIN NODE RESCHEDULE	Impact	Network isolates PG primary; risk of two leaders

Context: Network switch failure isolates salt-control-postgres-i1-hpkf-0 (primary). Replicas i2/i3 detect leader loss and elect a new primary. Old primary unaware of demotion.

Pre-Remediation Backups

#	Backup Action	Command
1	Snapshot etcd	<code>ETCDCTL_API=3 etcdctl snapshot save /backup/etcd-pre.db</code>
2	Backup PG cluster CR	<code>kubectl -n postgres-operator get perconapgcluster -o yaml > /backup/pg-cr.yaml</code>
3	Backup all namespace state	<code>kubectl get all -A -o yaml > /backup/cluster-state.yaml</code>
4	Verify pgBackRest backup	<code>kubectl -n postgres-operator exec <repo-host> -c pgbackrest -- pgbackrest info</code>
5	Trigger fresh PG backup	<code>kubectl -n postgres-operator create job --from=cronjob/<pgbackupmonitor> pre-maint-\$(date +%s)</code>

Remediation Steps

#	Action	Command / Detail
1	Detect partition	<code>kubectl get nodes grep NotReady</code>
2	Check Patroni on replica	<code>kubectl -n postgres-operator exec salt-control-postgres-i2-5tf6-0 -c database -- patronictl list</code>
3	Fence old primary node	<code>kubectl cordon <partitioned-node></code> <code>kubectl drain <partitioned-node> --force --grace-period=0</code>
4	Force-delete old primary	<code>kubectl -n postgres-operator delete pod i1-hpkf-0 --force --grace-period=0</code>
5	Prevent restart	<code>ssh <node> 'systemctl stop kubelet'</code>
6	Verify single primary	<code>psql -c 'SELECT pg_is_in_recovery();'</code> on new primary
7	Re-init old as replica	Delete pod, let operator recreate, verify replication
8	Verify data consistency	Compare <code>pg_current_wal_lsn</code> with <code>replay_lsn</code>

Post-Remediation Verification

#	Verification	Command	Expected
---	--------------	---------	----------

1	All nodes Ready	<code>kubectl get nodes</code>	<i>All STATUS=Ready</i>
2	All pods Running	<code>kubectl get pods -A grep -v Running grep -v Completed</code>	<i>No output</i>
3	PG primary writable	<code>psql -c 'SELECT pg_is_in_recovery();'</code>	<i>false</i>
4	Replication streaming	<code>psql -c 'SELECT state FROM pg_stat_replication;'</code>	<i>streaming</i>
5	PgBouncer connected	<code>SHOW POOLS;</code>	<i>Active > 0</i>
6	Backup healthy	<code>pgbackrest info</code>	<i>Recent backup</i>
7	Services up	<code>kubectl -n postgres-operator get svc</code>	<i>IPs assigned</i>
8	DaemonSets complete	<code>kubectl get ds -A</code>	<i>DESIRED = READY</i>
9	No split-brain	<code>patronictl list</code>	<i>Exactly 1 Leader</i>
10	Timeline consistent	<code>SELECT timeline_id FROM pg_control_checkpoint();</code>	<i>Same across cluster</i>

ALERT: Split-brain can cause data divergence. If both accepted writes, manual reconciliation with `pg_rewind` may be needed.

RESOLVED: Network restored. Old primary re-initialized as replica. Single leader confirmed.

SCENARIO 17: High kube-controller-manager Restarts (Cluster-B Observed)

Cluster	Cluster-B	Severity	P2 — High
Operations	REPLACE NODE	Impact	CM instability affects StatefulSet reconciliation for PG

Context: Observed: controller-manager restarts: 6p4tj=5, 8m9sf=11, qp7zx=9, wqt29=10, wr4gw=5. Scheduler also 4–10 restarts. Systemic control plane resource constraint.

Pre-Remediation Backups

#	Backup Action	Command
1	Snapshot etcd	<code>ETCDCTL_API=3 etcdctl snapshot save /backup/etcd-pre.db</code>
2	Backup PG cluster CR	<code>kubectl -n postgres-operator get perconapgcluster -o yaml > /backup/pg-cr.yaml</code>
3	Backup all namespace state	<code>kubectl get all -A -o yaml > /backup/cluster-state.yaml</code>
4	Verify pgBackRest backup	<code>kubectl -n postgres-operator exec <repo-host> -c pgbackrest -- pgbackrest info</code>
5	Trigger fresh PG backup	<code>kubectl -n postgres-operator create job --from=cronjob/<pgbackupmonitor> pre-maint-\$(date +%s)</code>

Remediation Steps

#	Action	Command / Detail
1	Check CM logs all CPs	<code>for cp in 6p4tj 8m9sf qp7zx wqt29 wr4gw; do kubectl -n kube-system logs kube-controller-manager-\$cp --previous --tail=30; done</code>
2	Check leader election	<code>kubectl -n kube-system get endpoints kube-controller-manager -o yaml</code>
3	Check resource usage	<code>kubectl top nodes grep control-plane</code>
4	Check etcd health	<code>etcdctl endpoint health --cluster</code>
5	If resource constrained	Increase VM CPU/memory for worst-affected CP nodes
6	Restart CM on worst	<code>ssh <node> 'mv /etc/kubernetes/manifests/kube-controller-manager.yaml /tmp/ && sleep 5 && mv /tmp/kube-controller-manager.yaml /etc/kubernetes/manifests/'</code>
7	Increase static pod limits	Edit kube-controller-manager.yaml: memory limits to 512Mi
8	Monitor restart count	<code>kubectl -n kube-system get pods grep controller-manager</code>

Post-Remediation Verification

#	Verification	Command	Expected
1	All nodes Ready	<code>kubectl get nodes</code>	<i>All Ready</i>
2	Pods healthy	<code>kubectl get pods -A grep -v Running grep -v Completed</code>	<i>Clean</i>
3	DaemonSets	<code>kubectl get ds -A</code>	<i>DESIRED = READY</i>
4	Services	<code>kubectl get svc -A</code>	<i>IPs assigned</i>
5	Events clean	<code>kubectl get events -A --sort-by=.lastTimestamp tail</code>	<i>No errors</i>
6	CM restarts stable	<code>kubectl get pods -n kube-system grep controller</code>	<i>No new restarts</i>
7	Scheduler stable	<code>kubectl get pods -n kube-system grep scheduler</code>	<i>No new restarts</i>

RESOLVED: Root cause: static pod memory limits too low. Increased to 512Mi across all CPs. Stable.

SCENARIO 18: PgBouncer Connection Saturation Under Load

Cluster	Cluster-A	Severity	P2 — High
Operations	RESIZE POD	Impact	Applications see 'too many connections' errors

Context: With only 2 PgBouncer replicas in Cluster-A, a traffic spike saturates connection limits. Applications receive connection refused errors.

Pre-Remediation Backups

#	Backup Action	Command
1	Backup PG cluster CR	<code>kubectl -n postgres-operator get perconapgcluster -o yaml > /backup/pg-cr.yaml</code>
2	Backup PgBouncer config	<code>kubectl -n postgres-operator get configmap -l app=pgbouncer -o yaml > /backup/pgb-cm.yaml</code>
3	Verify pgBackRest backup	<code>kubectl -n postgres-operator exec <repo-host> -c pgbackrest -- pgbackrest info</code>
4	Trigger fresh PG backup	<code>kubectl -n postgres-operator create job --from=cronjob/<pgbackupmonitor> pre-maint-\$(date +%s)</code>

Remediation Steps

#	Action	Command / Detail
1	Check PgBouncer stats	<code>psql -p 5432 -U pgbouncer pgbouncer -c 'SHOW POOLS;'</code>
2	Check config	<code>psql -p 5432 -U pgbouncer pgbouncer -c 'SHOW CONFIG;' grep -E 'max_client default_pool'</code>
3	Scale replicas via CR	<code>kubectl -n postgres-operator patch perconapgcluster pgkubcluster --type merge -p '{"spec":{"proxy":{"pgBouncer":{"replicas":4}}}}'</code>
4	Increase resources	Patch CR: <code>proxy.pgBouncer.resources.limits: cpu 2, memory 1Gi</code>
5	Tune settings	max_client_conn, default_pool_size via CR config
6	Verify new pods	<code>kubectl -n postgres-operator get pods -l postgres-operator.crunchydata.com/role=pgbouncer</code>
7	Verify balance	SHOW POOLS; on each PgBouncer pod, check cl_active

Post-Remediation Verification

#	Verification	Command	Expected
---	--------------	---------	----------

1	PG primary writable	<code>psql -c 'SELECT pg_is_in_recovery();'</code>	<i>false</i>
2	Replication streaming	<code>SELECT * FROM pg_stat_replication;</code>	<i>state=streaming</i>
3	Patroni cluster	<code>patronictl list</code>	<i>1 Leader + Replicas</i>
4	PgBouncer pools	<code>SHOW POOLS;</code>	<i>Active pool</i>
5	Backup health	<code>pgbackrest info</code>	<i>Within RPO</i>
6	PG services	<code>kubectl -n postgres-operator get svc grep postgres</code>	<i>primary + replicas</i>
7	4 PgBouncer pods	<code>kubectl get pods grep pgbouncer</code>	<i>4/4 Running</i>
8	Errors gone	App logs / metrics	<i>No connection refused</i>

RESOLVED: PgBouncer scaled 2 to 4 replicas. `max_client_conn` increased. Connection errors resolved.

SCENARIO 19: Cordon Node for Security Patch Without Downtime

Cluster	Both Clusters	Severity	P3 — Planned
Operations	DRAIN NODE RESCHEDULE	Impact	Kernel CVE patch; minimize PG impact

Context: Critical kernel CVE requires patching all worker nodes. Cluster-A (6 nodes, 2 PG instances) and Cluster-B (10 nodes, 3 PG instances) need sequential rolling patching.

Pre-Remediation Backups

#	Backup Action	Command
1	Snapshot etcd	<code>ETCDCTL_API=3 etcdctl snapshot save /backup/etcd-pre.db</code>
2	Backup PG cluster CR	<code>kubectl -n postgres-operator get perconapgccluster -o yaml > /backup/pg-cr.yaml</code>
3	Backup all namespace state	<code>kubectl get all -A -o yaml > /backup/cluster-state.yaml</code>
4	Verify pgBackRest backup	<code>kubectl -n postgres-operator exec <repo-host> -c pgbackrest -- pgbackrest info</code>
5	Trigger fresh PG backup	<code>kubectl -n postgres-operator create job --from=cronjob/<pgbackupmonitor> pre-maint-\$(date +%s)</code>

Remediation Steps

#	Action	Command / Detail
1	Map PG pods to nodes	<code>kubectl -n postgres-operator get pods -o wide grep -E 'i1 i2 i3 pgbouncer repo'</code>
2	Start with safe nodes	Nodes NOT hosting PG primary, PgBouncer, or repo-host
3	Cordon target	<code>kubectl cordon <node></code>
4	Drain with PDB respect	<code>kubectl drain <node> --ignore-daemonsets --grace-period=60</code>
5	Verify PDB exists	<code>kubectl -n postgres-operator get pdb</code>
6	Patch and reboot	<code>ssh <node> 'apt update && apt install linux-image-\$(uname -r) && reboot'</code>
7	Wait Ready, uncordon	<code>kubectl get nodes -w && kubectl uncordon <node></code>
8	Repeat for replica nodes	Drain replica node; verify replication catches up on new node
9	LAST: drain primary node	Triggers Patroni failover; verify new primary before proceeding
10	Full verification	<code>patronictl list && SELECT * FROM pg_stat_replication;</code>

Post-Remediation Verification

#	Verification	Command	Expected
1	All nodes Ready	<code>kubectl get nodes</code>	<i>All STATUS=Ready</i>
2	All pods Running	<code>kubectl get pods -A grep -v Running grep -v Completed</code>	<i>No output</i>
3	PG primary writable	<code>psql -c 'SELECT pg_is_in_recovery();'</code>	<i>false</i>
4	Replication streaming	<code>psql -c 'SELECT state FROM pg_stat_replication;'</code>	<i>streaming</i>
5	PgBouncer connected	<code>SHOW POOLS;</code>	<i>Active > 0</i>
6	Backup healthy	<code>pgbackrest info</code>	<i>Recent backup</i>
7	Services up	<code>kubectl -n postgres-operator get svc</code>	<i>IPs assigned</i>
8	DaemonSets complete	<code>kubectl get ds -A</code>	<i>DESIRED = READY</i>
9	Kernel patched	<code>ssh <node> uname -r</code>	<i>New kernel version</i>

WARNING: Patch PG primary node LAST. PodDisruptionBudgets prevent draining multiple PG instances simultaneously.

RESOLVED: All nodes patched and rebooted. PG cluster maintained availability throughout.

SCENARIO 20: Full DR: Rebuild PostgreSQL From Backup on New Nodes

Cluster	Cluster-A	Severity	P1 — Critical
Operations	DRAIN NODE REPLACE NODE RESIZE POD	Impact	Both PG instances lost; rebuild from pgBackRest backup

Context: Catastrophic storage subsystem failure causes PVC data loss on both nodes hosting pgkubcluster-i1-rvcz-0 and pgkubcluster-i2-d6hm-0. The repo-host pod (2/2 Running) still has pgBackRest backups accessible. Last successful backup was 17h ago.

Pre-Remediation Backups

#	Backup Action	Command
1	Snapshot etcd (cluster state)	<code>ETCDCTL_API=3 etcdctl snapshot save /backup/etcd-pre-dr.db</code>
2	Backup ALL CRs	<code>kubectl get all -A -o yaml > /backup/full-cluster-pre-dr.yaml</code>
3	Backup PG cluster CR	<code>kubectl -n postgres-operator get perconapgcluster -o yaml > /backup/pg-cr-dr.yaml</code>
4	CRITICAL: verify repo-host	<code>kubectl -n postgres-operator exec pgkubcluster-repo-host-0 -c pgbackrest -- pgbackrest info --stanza=db</code>
5	Note backup set ID	Record: full/incr backup timestamp, WAL archive status

Remediation Steps

#	Action	Command / Detail
1	Assess damage	<code>kubectl -n postgres-operator get pods -o wide</code>
2	Verify backup exists	<code>kubectl -n postgres-operator exec pgkubcluster-repo-host-0 -c pgbackrest -- pgbackrest info --stanza=db</code>
3	Drain failed nodes	<code>kubectl drain <node-1> --force --grace-period=0</code> <code>kubectl drain <node-2> --force --grace-period=0</code>
4	Provision replacements	IaC: 2 new workers with adequate storage
5	Join nodes	<code>kubeadm join <control-plane-endpoint></code>
6	Annotate CR for restore	<code>kubectl -n postgres-operator annotate perconapgcluster pgkubcluster --overwrite postgres-operator.crunchydata.com/pgbackrest-restore=id1</code>
7	Monitor restore progress	<code>kubectl -n postgres-operator logs <new-primary> -c pgbackrest -f</code>
8	Verify data integrity	<code>psql -c "SELECT count(*) FROM <critical_table>;"</code>
9	Verify replication	<code>psql -c 'SELECT * FROM pg_stat_replication;'</code>

10	Run post-restore backup	<code>kubectl -n postgres-operator create job --from=cronjob/<pgbackupmonitor> post-restore-\$(date +%s)</code>
11	Resize if needed	Patch CR for new node resource limits

Post-Remediation Verification

#	Verification	Command	Expected
1	Primary writable	<code>SELECT pg_is_in_recovery();</code>	<i>false</i>
2	Replica streaming	<code>SELECT state FROM pg_stat_replication;</code>	<i>streaming</i>
3	PgBouncer connected	<code>SHOW POOLS;</code>	<i>Active > 0</i>
4	Backup working	<code>pgbackrest info</code>	<i>Post-restore backup present</i>
5	All nodes Ready	<code>kubectl get nodes</code>	<i>All Ready</i>
6	All PG pods healthy	<code>kubectl get pods -n postgres-operator</code>	<i>All Running 5/5</i>
7	Services up	<code>kubectl -n postgres-operator get svc</code>	<i>primary + replicas + pgbouncer</i>
8	DaemonSets complete	<code>kubectl get ds -A</code>	<i>DESIRED = READY</i>
9	Monitoring active	Check Grafana PG dashboard	<i>Metrics flowing</i>
10	PMM connected	Check PMM UI	<i>All instances visible</i>

RESOLVED: Full restore from pgBackRest backup successful. RTO: 45 minutes. RPO: 17 hours (last backup).

22. QUICK REFERENCE COMMAND CHEAT SHEET

Node Operations

Operation	Command
Cordon (prevent scheduling)	<code>kubectl cordon <node></code>
Uncordon (allow scheduling)	<code>kubectl uncordon <node></code>
Drain (graceful)	<code>kubectl drain <node> --ignore-daemonsets --delete-emptydir-data --grace-period=60</code>
Drain (forced)	<code>kubectl drain <node> --force --grace-period=0 --timeout=120s</code>
Delete node object	<code>kubectl delete node <node></code>
Node conditions	<code>kubectl describe node <node> grep -A10 Conditions</code>
Resource usage	<code>kubectl top nodes</code>

PostgreSQL / Percona Operator

Operation	Command
List PG clusters	<code>kubectl -n postgres-operator get perconapgcluster</code>
Patroni cluster status	<code>kubectl exec <pg-pod> -c database -- patronictl list</code>
Check if primary	<code>psql -c 'SELECT pg_is_in_recovery();'</code>
Replication status	<code>psql -c 'SELECT * FROM pg_stat_replication;'</code>
Backup info	<code>kubectl exec <repo-host> -c pgbackrest -- pgbackrest info</code>
Manual failover	<code>kubectl exec <pod> -c database -- patronictl failover --force</code>
Trigger manual backup	<code>kubectl create job --from=cronjob/<cj> manual-\$(date +%s)</code>
Scale PgBouncer	<code>kubectl patch perconapgcluster <n> --type merge -p '{"spec":{"proxy":{"pgBouncer":{"replicas":N}}}}'</code>

Pod / Debugging

Operation	Command
Force delete pod	<code>kubectl delete pod <pod> --force --grace-period=0 -n <ns></code>
Pod resource usage	<code>kubectl top pod <pod> -n <ns> --containers</code>

Pods on specific node	<code>kubect1 get pods -A -o wide --field-selector spec.nodeName=<node></code>
Recent events	<code>kubect1 get events -n <ns> --sort-by=.lastTimestamp</code>
PodDisruptionBudgets	<code>kubect1 get pdb -n <ns></code>

23. ESCALATION MATRIX

Fill in team-specific contacts before distributing this runbook.

Severity	SLA	First Responder	Escalation	Examples
P1 Critical	15 min	On-call K8s SRE + DBA On-call	Eng Manager + VP Engineering	PG primary down, Node failures, Split-brain
P2 High	30 min	K8s SRE On-call	Team Lead	Replica degraded, Backup failures, OOM events
P3 Planned	Best effort	Assigned engineer	Team Lead	Node upgrades, Patching, Capacity changes

Post-Incident Review Requirements

All **P1** and **P2** incidents require a post-incident review within **48 hours**. The review must document: (1) Timeline of events with UTC timestamps, (2) Root cause analysis, (3) Impact assessment including any data loss, (4) Action items with owners and deadlines, (5) Updates to this runbook if new scenarios are discovered.